

Distributed Ride Hailing Backend System

Overview

A simplified ride-hailing backend system inspired by platforms like Uber and Ola. The project demonstrates event-driven architecture using Apache Kafka, backend API development with Node.js, MongoDB data storage, and containerized deployment using Docker.

The goal is to simulate the complete lifecycle of a ride request:

1. Rider requests a ride.
 2. Trip Service creates a trip.
 3. Kafka publishes a trip request event.
 4. Driver Assignment Service consumes the event.
 5. Available driver is assigned.
 6. Trip status is updated.
 7. Ride completion triggers payment processing.
-

Tech Stack

Backend

- Node.js
- Express.js

Database

- MongoDB
- Mongoose ODM

Messaging

- Apache Kafka
- KafkaJS

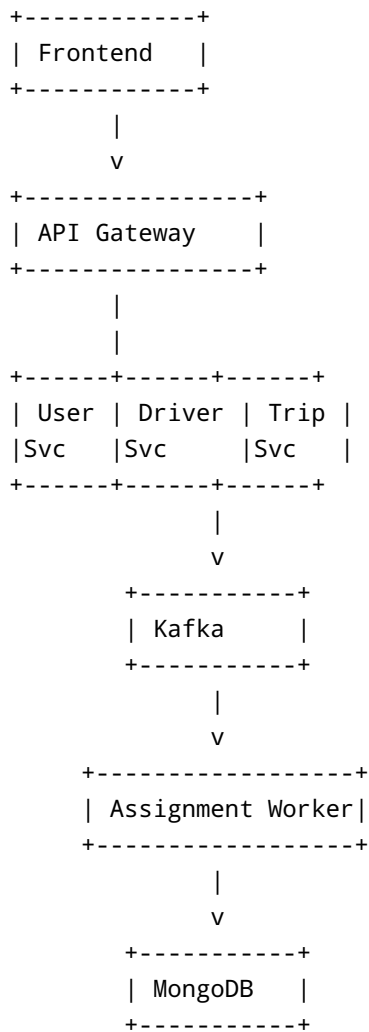
DevOps

- Docker
- Docker Compose

Authentication

- JWT (JSON Web Tokens)
 - bcrypt
-

System Architecture



Core Features

User Management

- User Registration
- User Login
- JWT Authentication
- User Profile

Driver Management

- Driver Registration

- Driver Availability Toggle
- Driver Location Tracking

Trip Management

- Create Ride Request
- Assign Driver
- Start Trip
- Complete Trip
- Trip History

Payment Processing

- Simulated Payments
- Payment Status Tracking

Event Driven Workflow

- Kafka Producer
- Kafka Consumer
- Asynchronous Driver Assignment

Database Schema

User

```
{
  _id,
  name,
  email,
  password,
  createdAt
}
```

Driver

```
{
  _id,
  name,
  available,
  location: {
    lat,
    lng
  }
}
```

```
}  
}
```

Trip

```
{  
  _id,  
  riderId,  
  driverId,  
  pickup,  
  destination,  
  status,  
  fare,  
  createdAt  
}
```

Payment

```
{  
  _id,  
  tripId,  
  amount,  
  status  
}
```

Kafka Topics

trip-requested

Published whenever a user requests a ride.

Example:

```
{  
  "tripId": "123",  
  "riderId": "456"  
}
```

driver-assigned

Published after successful driver assignment.

Example:

```
{
  "tripId": "123",
  "driverId": "789"
}
```

trip-completed

Published when a trip ends.

payment-processed

Published after payment completion.

API Endpoints

User Service

Register User

POST /api/users/register

Request:

```
{
  "name": "John",
  "email": "john@example.com",
  "password": "password123"
}
```

Response:

```
{
  "message": "User created"
}
```

Login

POST /api/users/login

Request:

```
{
  "email": "john@example.com",
  "password": "password123"
}
```

Response:

```
{
  "token": "jwt-token"
}
```

Driver Service

Register Driver

POST /api/drivers

```
{
  "name": "Mike"
}
```

Update Availability

PUT /api/drivers/:id/status

```
{
  "available": true
}
```

Trip Service

Create Trip

POST /api/trips

```
{
  "riderId": "123",
  "pickup": {
    "lat": 12.97,
    "lng": 77.59
  },
  "destination": {
    "lat": 12.99,
    "lng": 77.63
  }
}
```

Response:

```
{
  "tripId": "TRIP001",
  "status": "REQUESTED"
}
```

Get Trip

GET /api/trips/:id

Response:

```
{
  "tripId": "TRIP001",
  "status": "ASSIGNED",
  "driverId": "DR001"
}
```

Payment Service

Process Payment

POST /api/payments

```
{
  "tripId": "TRIP001",
```

```
"amount" : 250
}
```

Driver Assignment Workflow

Step 1

User requests a ride.

Trip Service stores trip.

Status:

```
REQUESTED
```

Step 2

Trip Service publishes:

```
trip-requested
```

event to Kafka.

Step 3

Assignment Worker consumes event.

Searches available drivers.

Step 4

Nearest available driver selected.

Driver status updated:

```
AVAILABLE -> BUSY
```

Step 5

Trip updated.

Status:

ASSIGNED

Step 6

Kafka publishes:

driver-assigned

event.

MongoDB Optimizations

Indexes:

```
DriverSchema.index({
  available:1
})

TripSchema.index({
  riderId:1
})

TripSchema.index({
  driverId:1
})

DriverSchema.index({
  location:"2dsphere"
})
```

Benefits:

- Faster rider trip history lookup
- Faster driver searches

- Efficient geospatial queries
-

Docker Deployment

Start all services:

```
docker-compose up --build
```

Services:

```
MongoDB
Kafka
Zookeeper
User Service
Driver Service
Trip Service
Payment Service
```

Testing

Manual Testing

1. Register Rider
 2. Register Driver
 3. Request Ride
 4. Verify Driver Assignment
 5. Complete Trip
 6. Process Payment
-

Load Testing

Tool:

```
k6
```

Metrics:

- Request Throughput
- Response Time

- Error Rate
-

Future Enhancements

- Redis Caching
 - API Gateway
 - Rate Limiting
 - OpenTelemetry Tracing
 - Kubernetes Deployment
 - CI/CD Pipeline
 - Surge Pricing
 - Real-Time Driver Tracking
 - WebSocket Notifications
-

Learning Outcomes

This project demonstrates:

- Microservice Architecture
 - Event-Driven Systems
 - Kafka Messaging
 - MongoDB Schema Design
 - Docker Containerization
 - JWT Authentication
 - Distributed Backend Concepts
 - System Design Fundamentals
-

Resume Description

Built a distributed ride-hailing backend system using Node.js, MongoDB, Apache Kafka, and Docker. Implemented event-driven communication for ride request processing and driver assignment workflows. Designed optimized database schemas and indexing strategies to improve query performance under simulated high-load conditions. Containerized services for consistent deployment and scalability testing.